Implementation of Floating Point Unit based on Booth Multiplier and Compressor Adder

D.Vaithiyanathan
Department of Electronics and Communication Engineering
National Institute of Technology
Delhi, India
dvaithiyanathan@nitdelhi.ac.in

J.Britto Pari
Department of Electronics and Communication Engineering
Sri Sairam Engineering College, Chennai, India
brittopari@yahoo.co.in

S. Keerthana
Department of Electronics and Communication Engineering
College of Engineering Guindy,
Chennai, India
keerthanask94@gmail.com

Kala Bharathan
Department of Electronics and Communication Engineering
PES University
Bangalore, India
kala@pes.edu

***Abstract*— High speed multipliers plays a major role in digital signal processing applications, banking, tax and currency conversions, radar navigation and in real time 3D graphics display. Various techniques have been proposed to design multipliers which are efficient in terms of speed, area and power. Floating point multiplier is better than fixed point multiplier because of wide dynamic range. The floating point multiplier with 3:2 compressor adder and modified booth algorithm is more efficient when compared with other multipliers. Booth technique with compressor adder is used to reduce the partial product in order to increase the speed. This work is implemented in XILINX ISE simulator where the delay and area is improved by 54% and 60.47% in 3:2 compressor adder with modified booth based FPM when compared with compressor adder with modified booth based Floating Point Multiplier, 50% and 40% when compared with compressor adder with Wallace based Floating Point Multiplier and compressor adder based Floating Point Multiplier. Hence speed and area utilized is optimized in 3:2 compressor adder with floating point Multiplier.**

Keywords—Floating point multiplier, Adder, Compressor adder, Wallace Tree, Booth multiplier, FPGA

I. INTRODUCTION

In the modern world, overall performance of a processor is determined only by the adder and multiplier circuit. At the same time most of the problems in the real world are very complex in nature, When compared with the fixed point (FP) multiplier, floating point multiplier is best suited for high precision. But floating point computation is very complex in nature and to store and representation. At the same time most of the scientific computation widely use floating point numbers and customizing those application either in embedded system or hardware implementation such as field programmable gate arrays (FPGAs) devices.

Many authors reported their research work on these circuit in order to improve the VLSI performance matrices such as area and power optimization and to reduce the delay [1-13]. Sunesh and Sathiskumar [1] proposed the fast floating point multiplier unit (FPU), they implemented by pipelining Kogge stone adder for partial product addition and in multiplier used the radix 4 booth encoder and arranged the partial product in Wallace tree structure. In that work the speed is improved to a great extent than other multipliers but the power consumption is high. Wai-leong pang et al., [2] proposed the radix 4 booth multiplier for the 24-bit mantissa generation which is later combined with sign, exponent and mantissa for the complete FPM. In this paper author achieved the power reduction and increased the speed by using the Wallace tree structure but fails to reduce the area. Saha et al.,[3] proposed the improved floating point multiplier (FPM) design based on canonical sign digit (CSD) and the partial product are generated using baugh wooley's multiplier where the speed and power are improved drastically but circuit occupies more area compared with conventional multiplier. Mohamad al et al., [4] reported that the FPM does not need rounding for significant multiplicationand when used as the complete 48-bits the performance would be better. In their work author used the carry save adder and moreover FPM is analysed together with the pipelining and achieved the improvement in speed but inefficient in terms of power and area. Pallavi Ramteke et el., [5] proposed the floating point multiplier has been implemented using shift and add algorithm which reduces the execution time of the multiplier. In this rounding is not preferred and direct multiplication of the significant is used in order to improve the speed, though the delay is reduced, area and power are not optimized. An efficient single precision floating point multiplier architecture based on classical recoding algorithm was proposed in the year 2016 by Jean Jenifer Nesam and Sivanantham Sathasivam [6]. In this work author explained about the parallel processing of the floating point multiplied by divide the data into the different blocks and perform the operation. Partial product generation is done with modified booth algorithm and final addition with the use of mixed carry select adder and carry save adder of the operation with low thermal power performance. In [6] delay is highly reduced but area is not optimized. Inference from the literature in our implementation of FPU, we going to use the 3:2 compressor adder with modified booth based FPM, where compressor adder is formed with XOR-XNOR MUX in order to reduce the gate count and modified booth to reduce the partial product stages.

II. BACKGROUND

A. Floating Point Representation

Unlike the integer number representation, floating point representation have standard format which recognized by the institute of Electrical and Electronics Engineers called as IEEE-754 standard. Table 1 shows the different representation of IEEE-754 format of floating point numbers. Though they are different types of representation for a floating numbers, it can be selected based on the application. Due the memory constraint, complexity of the hardware, area, speed and power today most of the real time application are using the single precision representation. Only for higher precision application user can prefer double or quadruple precision with the cost of memory and hardware complexity.

B. Floating Point Computations

Floating point computation are performed with the computation units which are similar to the normal integer styled arithmetic units with slight modifications in

978-1-5386-8158-9/19/$31.00 ©2019 IEEE

Authorized licensed use limited to: LAHORE UNIV OF MANAGEMENT SCIENCES. Downloaded on November 03,2026 at 07:27:21 UTC from IEEE Xplore. Restrictions apply.

computation logics as shown in Figure 1. As the floating point representation consists three individual parts, individual computation procedures are requisite for each part. Though there are three individual computations are employed, most of the arithmetic complexity restrain within the mantissa operations itself. As the mantissa terms involves the real arithmetic computations, and the computations are performed with respect to its sign and exponent parts. For e.g., the floating point adders uses any of the adder structures like carry look ahead adder, carry save adder, carry bypass adder or ripple carry or any other adder to add the mantissa terms, but the addition have to be done in accordance with the sign and exponent bits. Let X_1 (s_1,e_1,m_1) and $X_2(s_2,e_2,m_2)$ be two floating point numerals. Then $X_3(s_3,e_3,m_3)$ be the addition result of X_1 and X_2, the computation is done basically by two steps [7].

TABLE I. REPRESENTATION OF IEEE-754 FORMAT OF FLOATING POINT NUMBER

Representation	Total Bits	Sign Bit	Mantissa Bits	Exponent Bits
Half precision	16	1	10	5
Single Precision	32	1	23	8
Double Precision	64	1	52	11
Quadruple precision	128	1	112	15

$$s_3 = s_1 \text{ xor } s_2 \quad (1)$$

$$e_3 = \max\{e_1,e2\}; \quad d = e_1 \sim e_2 \quad (2)$$

$$m_3 = \{m_1\}_d \{\pm\}_{s3} \{m_2\}_d \quad (3)$$

The sign bit chooses the operations weather addition or subtraction, whereas the exponent decides the operands in mantissas to which is to be added. These shows the whole addition process apart from adding also includes some extra logics, which increases the computation complexity. This will become worse in the case of multiplication operation in which the random multiplication patterns have to be done on the mantissa parts with respect to their exponent parts, Let $X_4(s_4, e_4, m_4)$ be the output of the floating point numbers for two inputs X_1 and X_2, then the computation done as follows [7-8]

$$s_4 = s_1 \,\&\, s_2 \quad (4)$$

$$e_4 = e_1 + e_2 \quad (5)$$

$$m_3 = \{m_1\}_{<<e1} * \{m_2\}_{<<e2} \quad (6)$$

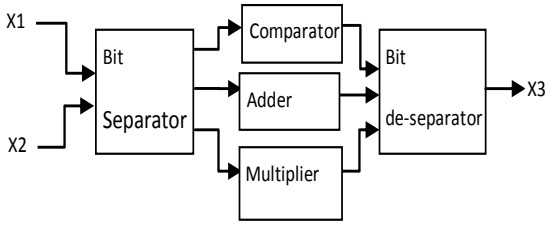


Figure 1. Conventional Floating Point Multiplier [7]

III. IMPLEMENTATION OF FLOATING POINT UNIT

Power, delay and area product plays a major in all the arithmetic units but it is observed with the existing multipliers the issue is high power and area. In order to avoid the disadvantage of high power, area and delay an optimized circuit is needed. Hence 3:2 Compressor adder with Modified Booth based FPM is implemented and simulated in order to improve the speed and reduce the area. Modified Booth algorithm is used to reduce the number of stages with partial product generation which results speed, area and power optimization. When only 3:2 compressor adder is used with Modified Booth Algorithm will increase the speed of the multiplier. Compressor adder is used as it will reduce the delay to a great extent. As 3:2 compressor adder is the basic unit which can replace the full adder and produce less gate count. By keeping 3:2 compressor adder as the basic one it is possible to generate N:2 compressor adder. Here 3:2 to 24:2 compressor adders are generated. The main advantage of compressor adder is to reduce the delay.

A. Compressor Adder

It is the combinational circuit to perform addition of more than 3-bits with lesser number of gate counts and delay when compare to the equivalent full adder circuit. Since the compressor adder satisfies almost all the VLSI performance parameter it is most widely used for multipliers to image and signal processing applications. Figure 2 describes the 3:2 compressor adder and Figure 3 Architecture of 3:2 compressor adder where sum and carry are described in equation 7- 8 and Figure 4 described 4:2 compressor adder and Figure 5 Architecture of 4:2 compressor adder which describes the use of 3:2 compressor adder in 4:2 compressor adder and sum, carry, cout are given in the equations 9-11.

$$\text{Sum} = (A\hat{}B)*\sim C + (\sim (A\hat{}B))*C \quad (7)$$

$$\text{Carry} = (A\hat{}B)*C + (\sim (A\hat{}B))*A \quad (8)$$

$$\text{Sum} = A \hat{} B \hat{} C \hat{} D \hat{} \text{Cin} \quad (9)$$

$$\text{Cout} = (A\hat{}B)*C + (\sim (A\hat{}B))*A \quad (10)$$

$$\text{Carry} = (A\hat{}B\hat{}C\hat{}D)* \text{Cin} + (\sim (A\hat{}B\hat{}C\hat{}D))*D \quad (11)$$

As the 3:2 compressor adder will replace the full adder by XOR gate and multiplexer where the performance will be same but gate count get reduced. On reducing the gate count, the delay gets decreased. For 4:2 compressor adder usage of two 3:2 compressors are preferred. Likewise 24:2 compressor adder is designed with the use of 3:2 compressor adder. Compressor adder is used to reduce the N number of inputs into single Sum and Carry with (N-3) Cout. Cout in the compressor adder will increase the multiplier size moderately but the delay of the circuit gets reduced to greater extent. Since the modified booth and Wallace tree multiplier are used for long time, about these multiplier is not explained even though we are using these methods in our FPM implementation.

Authorized licensed use limited to: LAHORE UNIV OF MANAGEMENT SCIENCES. Downloaded on November 03,2026 at 07:27:21 UTC from IEEE Xplore. Restrictions apply.

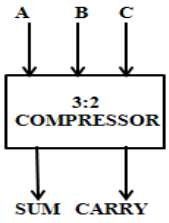


Figure 2. 3:2 Compressor Adder

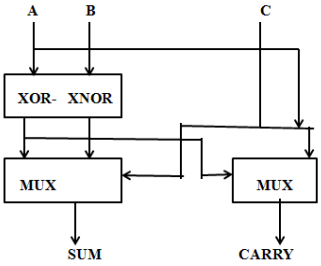


Figure 3. Architecture of 3:2 Compressor Adder

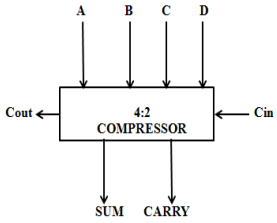


Figure 4. 4:2 Compressor Adder

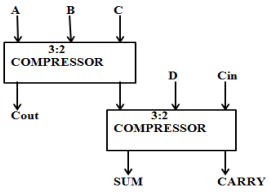


Figure 5. Architecture of 4:2 Compressor Adder

B. Multiplier for Mantissa Generation

1) Only 3:2 Compressed Adder with Modified Booth Multiplier: Partial products for the multiplier are generated using modified booth technique and accumulation of partial products are performed by only 3:2 compressor adder in each stages of reduction will reduce the execution time of the operation. The final addition also completed only with 3:2 compressor adder and half adder. By combining 3:2 compressor adder with modified booth encoder will reduce the delay of 24-bit mantissa calculation in the floating point multiplier. If the size of the compressor adder is extend further the delay will reduce further. The architecture of only 3:2 compressor adder with modified booth based FPM is shown in Figure 6 [1-6].

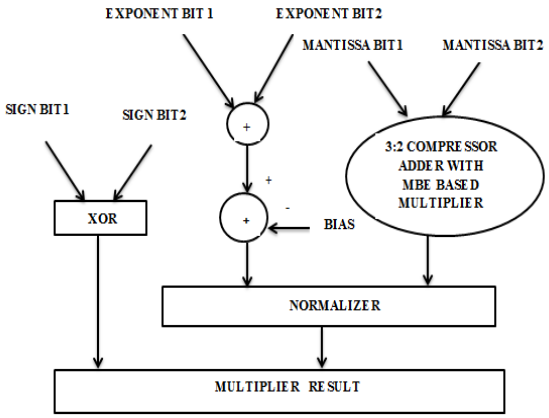


Figure 6 Architecture of only 3:2 compressor adder with Modified Booth based FPM

2) Compressed Adder with Wallace tree Multiplier: Only difference between this method and the early one is the partial product are arranged in Wallace tree structure in order to increase the speed with optimized area. The architecture of compressor adder with Wallace based FSM is shown in Figure 7.

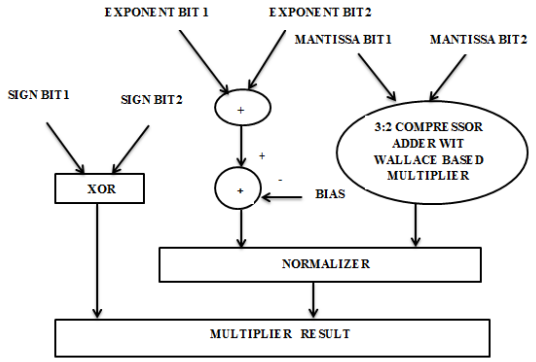


Figure 7 Architecture of Compressor adder with Wallace based FPM

IV. IMPLEMENTATION RESULTS AND EVALUATION

The 3:2 Compressor adder with modified booth based FPM is compared with compressor adder with modified booth based FPM, compressor adder with Wallace based FPM. The Compressor adder based FPM are described using Verilog HDL(Hardware Description Language) and simulation and synthesized in Xilinx ISE using Virtex6-XC6VCX75T field programmable gate array (FPGA) device. The simulated waveforms are shown in Figure 8 -14. The propagation delay for compressor adder based FPM is compared with Compressor adder with Wallace based FPM, Compressor adder with modified booth based FPM is tabulated in Table 2 and 3 and the graphical representations are given in the Figure 15-16.

A. Delay Analysis

Delay of mantissa part in FPM is discussed in the Table 2. Delay is improved to 63% when compared with compressor adder with Modified based Multiplier, 65% when compared with compressor adder with Wallace based Multiplier, 65.97% when compared with compressor adder based Multiplier.

From the Table 3 it is observed that the combinational delay of 3:2 compressor adder with modified booth based FPM is very small when compared with compressor adder based FPM, compressor adder with Wallace based FPM and compressor adder with modified booth based FPM. Delay is improved to 54% when compared with compressor adder with modified booth based FPM, 50.26% compared with compressor adder with Wallace based FPM and 50.16% compared with compressor adder based FPM.

TABLE II. RESULT ANALYSIS OF MANTISSA PART IN FPM

Methods	Total Delay (ns)	Gate Count	Area Utilization (%)
Compressor Adder With only 3:2 Compressor Adder With Modified Booth Based Multiplier	16.168	775	40
Compressor Adder With Modified Booth Based Multiplier	44.231	787	40
Compressor Adder With Wallace Based Multiplier	47.517	1259	40
Compressor Adder Based Multiplier	47.516	1259	40

Authorized licensed use limited to: LAHORE UNIV OF MANAGEMENT SCIENCES. Downloaded on November 03,2026 at 07:27:21 UTC from IEEE Xplore. Restrictions apply.

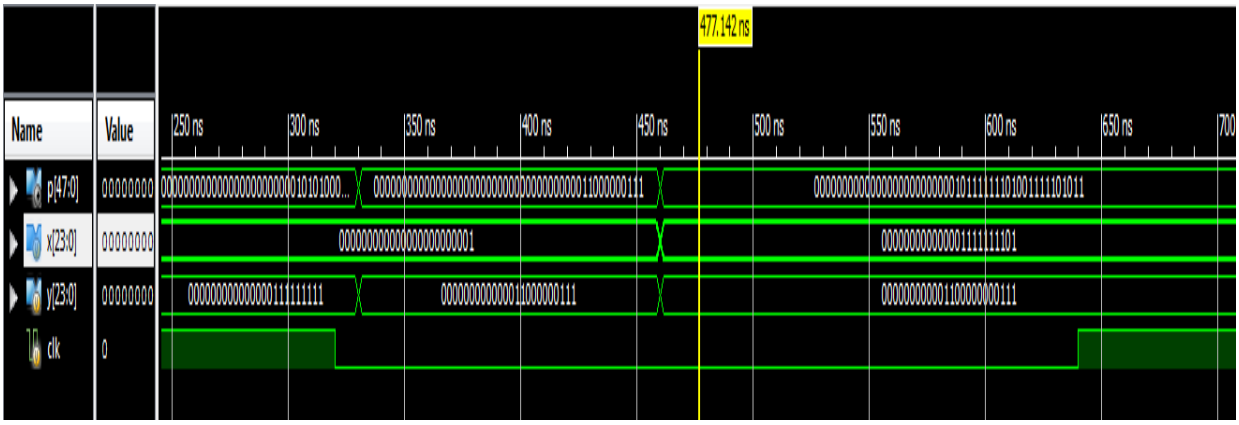


Figure 8 Output of 3:2 Compressor adder with modified booth multiplier

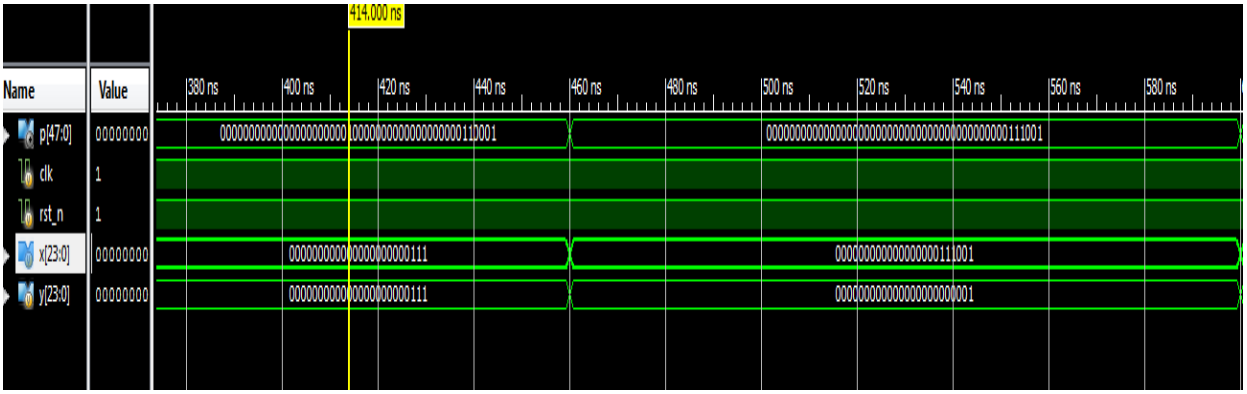


Figure 9 Output of 3:2 to 24:2 Compressor adder with modified booth multiplier

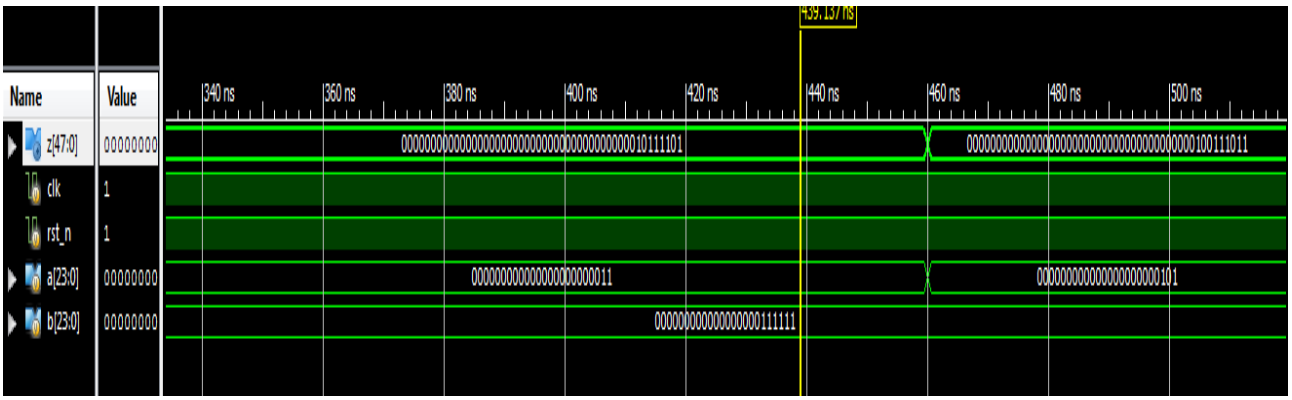


Figure 10 Output of 3:2 to 24:2 Compressor adder with Wallace multiplier

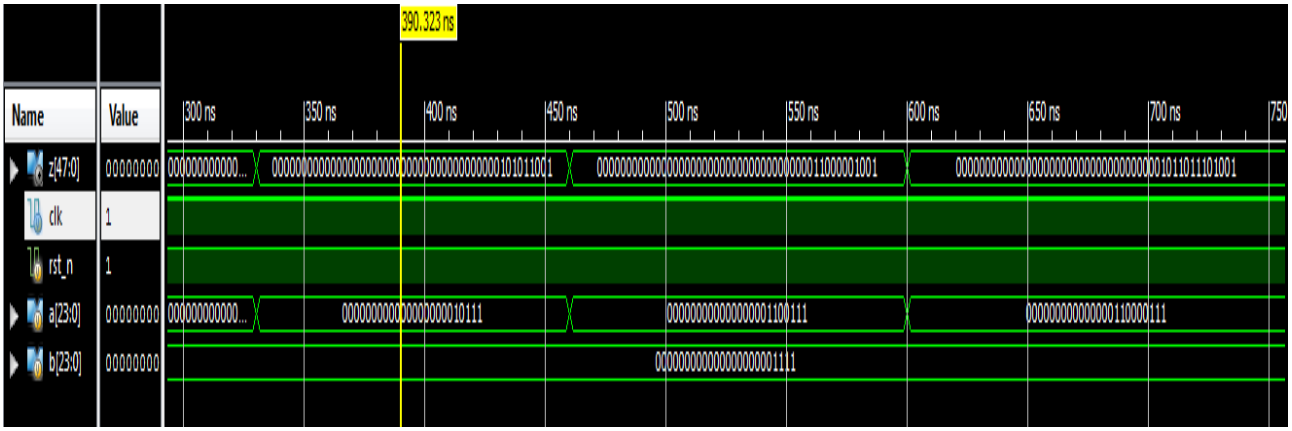


Figure 11 Output of 3:2 to 24:2 compressor adder based multiplier

Authorized licensed use limited to: LAHORE UNIV OF MANAGEMENT SCIENCES. Downloaded on November 03,2026 at 07:27:21 UTC from IEEE Xplore. Restrictions apply.

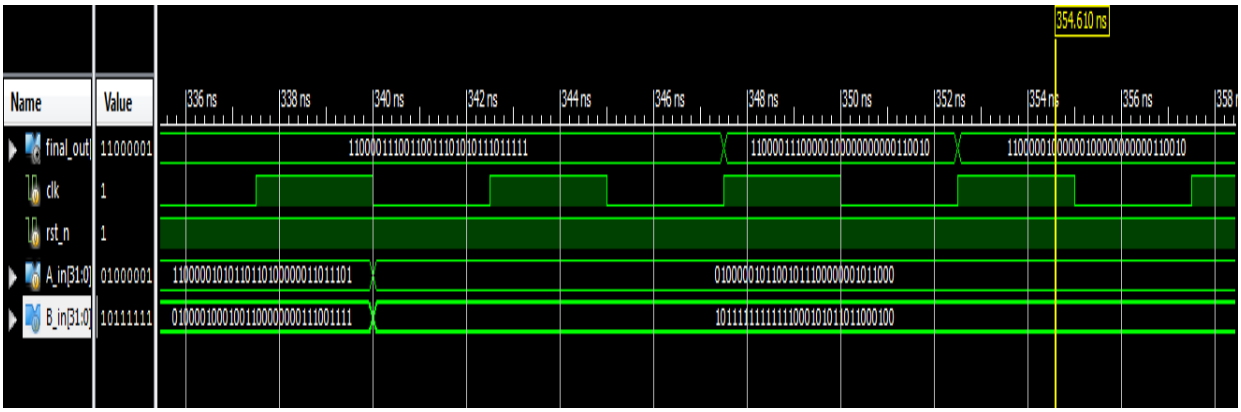


Figure 12 Output of 3:2 Compressor adder with modified booth based FPM

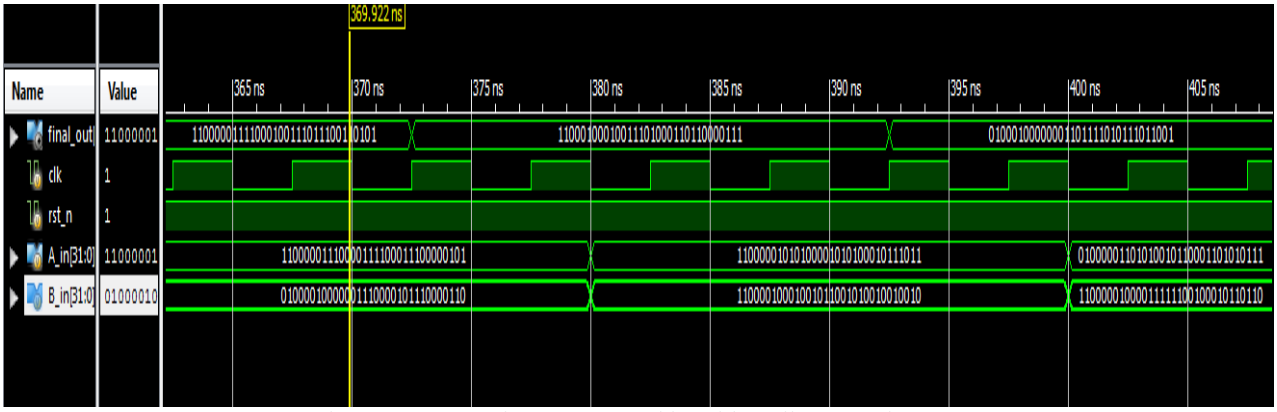


Figure 13 Output of Compressor Adder with Wallace Based FPM

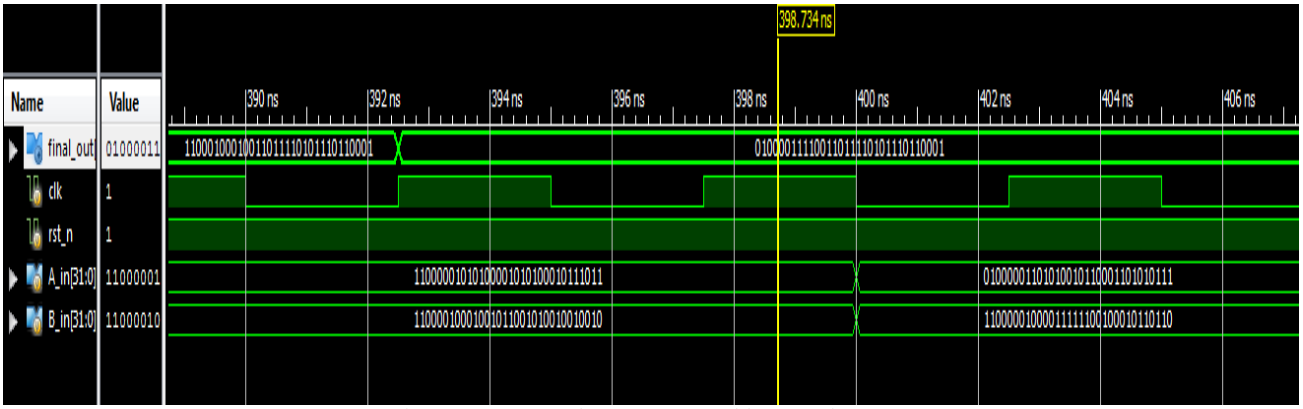


Figure 14 Output of Compressor Adder Based FPM

B. Area Analysis

Gate count for mantissa part is given in the Table 2. It is observed that the gate count for only 3:2 compressor adder for Modified Booth based Multiplier is improved to 15% when compared with compressor adder with Modified Booth based Multiplier, 38% when compared to compressor adder with Wallace based Multiplier and compressor adder based Multiplier.

From the Table 3it is observed that the gate count for 3:2 compressor adder based FPM improved to 60.47% in 3:2 compressor adder with modified booth based FPM when compared with compressor adder with modified booth based FPM, 40.05% when compared with compressor adder with Wallace based FPM and 40.09% when compared with compressor adder based FPM.

TABLE III. RESULT ANALYSIS OF DIFFERENT TYPES OF FLOATING POINT MULTIPLIER

Methods	Total Delay (ns)	Frequency (MHz)	Gate Count	Area (%)
Compressor Adder With Only 3:2 Compressor Adder With Modified Booth Based Floating Point Multiplier	11.032	90.644	901	40
Compressor Adder With Modified Booth Based Floating Point Multiplier	20.560	48.639	959	40
Compressor Adder With Wallace Based Floating Point Multiplier	22.183	45.079	1503	40
Compressor Adder Based Floating Point Multiplier	22.153	45.177	1504	46

Authorized licensed use limited to: LAHORE UNIV OF MANAGEMENT SCIENCES. Downloaded on November 03,2026 at 07:27:21 UTC from IEEE Xplore. Restrictions apply.

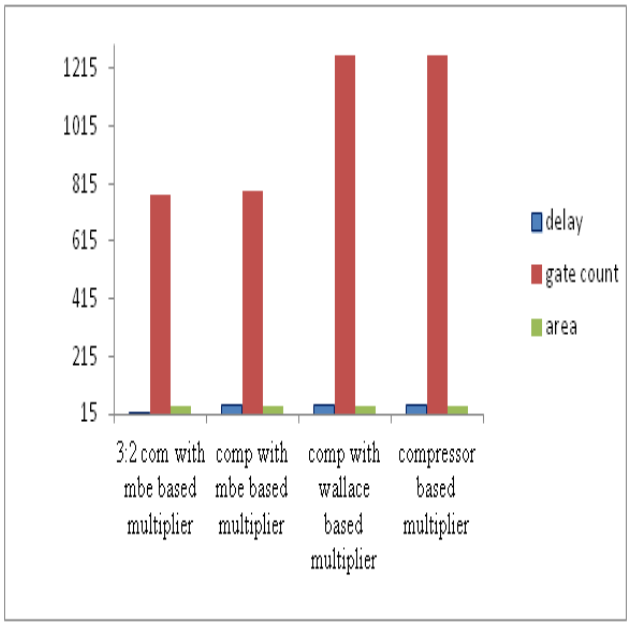


Figure 15 Comparison of Mantissa

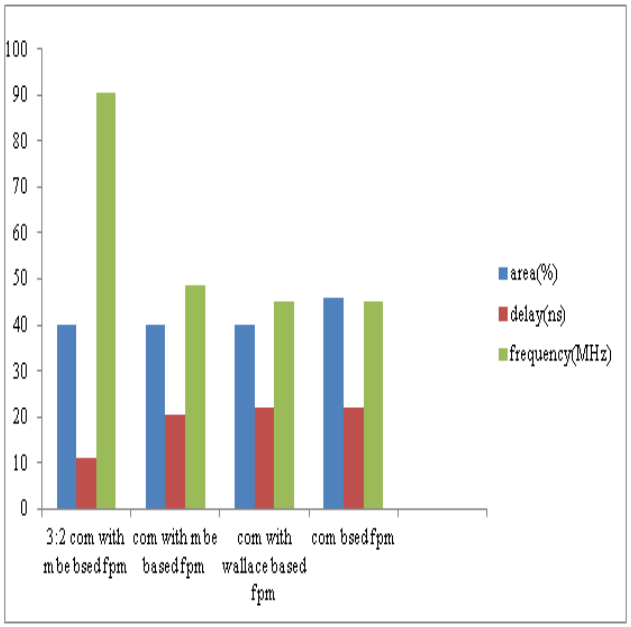


Figure 16 Comparison between types of FPM

C. *Frequency Analysis*

It is observed from the Table 3 that 3:2 compressor adder with modified booth based FPM is maximum operated at 90.644 MHz, compressor adder based FPM is minimum operated at 45.177MHz. The frequency operated for 3:2 compressor adder with modified booth based FPM is maximum due to the operation of modified booth algorithm and compressor adder.

From the above result it is inferred that the 3:2 compressor adder with modified booth based FPM is more efficient in terms of speed and area when compared with other architectures like compressor adder with modified booth based FPM, compressor adder with Wallace based FPM and compressor adder based FPM.

V. Conclusion

Thus, the implementation of 3:2 compressor adder with modified booth based FPM is more efficient when compared with other multipliers like compressor adder with modified booth based FPM, Compressor adder with Wallace based FPM and Compressor adder based FPM. From the result analysis it is observed that the delay and area of 54% and 60.47% is improved in 3:2 compressor adder with modified booth based FPM when compared with compressor adder with modified booth based FPM, 50% and 40% when compared with other type multipliers. In future, compressor adder based FPM can be implemented efficiently by reducing the delay in mantissa part generation. Mantissa part in FPM can also be implemented with other type of multipliers in order to reduce the overall delay, area and power. It can also be implemented for double precision FPM in future.

References

[1] Sunesh .N.V, Sathiskumar.P, “Design and implementation of fast floating point multiplier unit”, In Proceedings of VLSI Systems, Architecture, Technology and Applications (VLSI-SATA), Banglore Inia, Jan.2015.

[2] Wai-leong Pang, Kah-yoong Chan, Sew-kin Wong, Choon-siang Tan, “VHDL Modeling of Booth Radix-4 Floating Point Multiplier for VLSI Designer’s Library”, WSEAS Transactions on Systems, Issue.12, Vol.12, pp. 678-688, December 2013

[3] P. Saha, P. Bhattacharyya, A. Dandapat, “Improved Floating Point Multiplier Design Based on Canonical sign Digit”, In Proceedings of Journal of Technology, Vol. 5, No. 1, pp. 22-31, 2018.

[4] Mohamad Al.Ashrafy, Ashraf salem, Wagdy Anis, “An Efficient Implementation of Floating Point Multiplier”, In Proceeding of Saudi International Electronics, Communication and Photonics (SIECPC) , Riyadh, Saudi Arabia, April 2011.

[5] Pallavi Ramteke, N. N. Mhala, P. R. Lakhe, “Single Precision Floating Point Multiplier Using Shift And Add Algorithm”, International Journal of Innovative Research In Advanced Engineering (IJIRAE), Vol. 1, Issue 5, pp. 184-187, June 2014.

[6] J. Jean Jenifer Nesam and Sivanantham Sathasivam, “An Efficient Single Precision Floating Point Multiplier Architecture based on Classical Recoding Algorithm”, Indian Journal of Science and Technology, Vol. 9, Issue.5, February 2016.

[7] Harish Anand T, D.Vaithiyanathan, R.Seshasayanan, “Optimized Architecture for Floating Point Computation Unit”, In Proceedings of Emerging Trends in VLSI, Embedded Systems, Nano Electronics and Telecommunication Systems, Tiruvannamalai, India, January 2013

[8] Vaithiyanathan, D., and R. Seshasayanan. "An efficient low power log based FPU design for FPGAs." Advances in Natural and Applied Sciences, vol. 9, no. 6, 2015

[9] Mark G.Arnold, “A Real/Complex Logarthamic Number System ALU”, IEEE Transactions on Computers, vol.60, No. 02, pp. 202-213, Feb 2011.

[10] Mark A. Erle, Brian J. Hickmann and Michael J. Schulte, “Decimal Floating-Point Multiplication”, IEEE Transactions on Computers, Vol. 58, No. 7, July 2009.

[11] Jung-Yup Kang, Jean-Luc Gaudiot, “A Simple High Speed Multiplier”, IEEE Transactions on Computers, Vol. 55, No. 10, October 2006.

[12] Wang JP, Kuang SR, Liang SC, “High-Accuracy Fixed-Width Modified Booth Multipliers for Lossy Applications”, IEEE Transactions on Very Large Scale Integration (VLSI) Systems, Vol.19, No.1, 52- 60, January 2011

[13] P.S. Tulasiram, D.Vaithiyanathan and R.Seshasayanan, "Implementation of Modified Booth Wallace Tree multiplier for fast Arithmetic Circuits", International Journal of Advanced Research in Computer Science and Software Engineering, Vol. 4, No. 10, pp. 798 - 802, 2014

Authorized licensed use limited to: LAHORE UNIV OF MANAGEMENT SCIENCES. Downloaded on November 03,2026 at 07:27:21 UTC from IEEE Xplore. Restrictions apply.